

OBSERVATION TASK-5

Course:Full Stack Web Development

Course Code:23CM421

Experiment No:5-2

Student Name:R.Mohini

Roll No:A24126552110

1. TITLE

Node.js File Management Application

2. AIM

To develop a Node.js application that accepts a filename and content from the user and performs file operations — create/write, read, append, and display final contents — using the File System (fs) module.

3. OBJECTIVE

To create and write content to a file using `fs.writeFileSync()`.

This document doesn't have any headings. To add headings to your Table of Contents, go to Home > Styles

- To read the contents of a file using `fs.readFileSync()`.
- To append additional content to an existing file using `fs.appendFileSync()`.
- To display the final updated contents of the file after all operations.
- To accept user input interactively using the `readline` module.
- To build a simple, interactive Node.js File Management application.

4. THEORY

- Node.js is a JavaScript runtime built on Chrome's V8 engine that allows JavaScript to run outside the browser, enabling server-side and file-system operations.
- The File System (fs) module is a built-in Node.js module used to interact with the file system — creating, reading, updating, and deleting files.
- `fs.writeFileSync()` creates a new file (or overwrites an existing one) with the given content.
- `fs.readFileSync()` reads the contents of a file synchronously.
- `fs.appendFileSync()` appends additional content to an existing file without overwriting it.

- The readline module allows a Node.js console application to accept input from the user interactively.

Flow of application:

User Input (filename & content) → fs.writeFileSync() (create/write) → fs.readFileSync() (read) → fs.appendFileSync() (append) → fs.readFileSync() (display final content)

5. ALGORITHM / PROCEDURE

1. Import the fs and readline modules.
2. Create a readline interface to accept input from the console.
3. Prompt the user to enter a filename.
4. Prompt the user to enter the initial content for the file.
5. Use fs.writeFileSync() to create the file and write the initial content.
6. Read and display the file contents using fs.readFileSync().
7. Prompt the user to enter additional content to append.
8. Use fs.appendFileSync() to append the new content to the file.
9. Read and display the final updated content of the file using fs.readFileSync().
10. Close the readline interface and terminate the program.

6. PROGRAM

```
// fileManager.js
const fs = require('fs');
const readline = require('readline');

const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout
});

rl.question('Enter the file name: ', (fileName) => {
  rl.question('Enter the content to write: ', (content) => {
    // Create / Write the file
    fs.writeFileSync(fileName, content);
    console.log('\nFile created and content written successfully.');
```

```
    // Read the file
    let data = fs.readFileSync(fileName, 'utf8');
    console.log('\n--- File Content After Writing ---');
    console.log(data);
```

```

    rl.question('\nEnter additional content to append: ',
(appendContent) => {
    // Append content
    fs.appendFileSync(fileName, '\n' + appendContent);
    console.log('\nContent appended successfully.');
```

```

    // Read final content
    let finalData = fs.readFileSync(fileName, 'utf8');
    console.log('\n--- Final File Content ---');
```

```

    console.log(finalData);

    rl.close();
  });
});
});
```

7. CODE EXPLANATION

Code	Explanation
require('fs')	Imports Node.js's built-in File System module.
require('readline')	Imports the module used to accept input from the console.
readline.createInterface()	Creates an interface to read user input line by line.
rl.question()	Displays a prompt and captures the user's typed input.
fs.writeFileSync()	Creates the file (or overwrites it) and writes the initial content.
fs.readFileSync()	Reads and returns the current contents of the file.
fs.appendFileSync()	Appends new content to the end of an existing file.
rl.close()	Closes the readline interface and ends the program.

8. TEST CASES AND EXECUTION DATA

Test Case	Input	Expected Result	Actual Result	Status
TC-01	File: notes.txt, Content: "Hello World"	File created and written	Works correctly	Pass
TC-02	Read immediately after writing	Displays exact written content	Works correctly	Pass
TC-03	Append: "This is appended text"	New content added after existing	Works correctly	Pass

Test Case	Input	Expected Result	Actual Result	Status
TC-04	Read after append	Displays combined (original + appended) content	Works correctly	Pass
TC-05	Filename left unchanged, run program twice	Second run overwrites, then appends fresh content	Works correctly	Pass

9. OUTPUT:

Sample console execution:

```
$ node fileManager.js
Enter the file name: notes.txt
Enter the content to write: Hello, this is my first file in
Node.js!

File created and content written successfully.

--- File Content After Writing ---
Hello, this is my first file in Node.js!

Enter additional content to append: This line was appended later.

Content appended successfully.

--- Final File Content ---
Hello, this is my first file in Node.js!
This line was appended later.
```

10. RESULT

Thus, a Node.js File Management Application was successfully developed using the File System (fs) module. The application demonstrates file creation, writing, reading, appending, and displaying operations, along with interactive user input handled through the readline module.